

Multimodal Models 2 - Cross-modal Approaches

Petra Galuscakova

petra.galuscakova@uis.no



Department of Electrical Engineering and Computer Science
University of Stavanger

March 5, 2026



DALL.E

Imagen

Stable Diffusion

Flamingo



- ▶ Multimodal training: training generative models to convert between two or more different kinds of data
- ▶ E.g. text-to-image generation focuses on producing images from a given text prompt.
- ▶ Multimodal modeling presents an additional challenge to purely image processing and text processing, because the model must also learn how to **cross the bridge between the two domains**.
- ▶ The model must be also able to combine concepts and styles that it may never have seen before.
- ▶ Equally, it would be desirable for the model to accurately infer how objects in the generated image relate to each other, based on the text prompt (e.g. “astronaut riding a doughnut through space” vs. a “astronaut eating a doughnut in a crowded space”.)



DALL.E

Imagen

Stable Diffusion

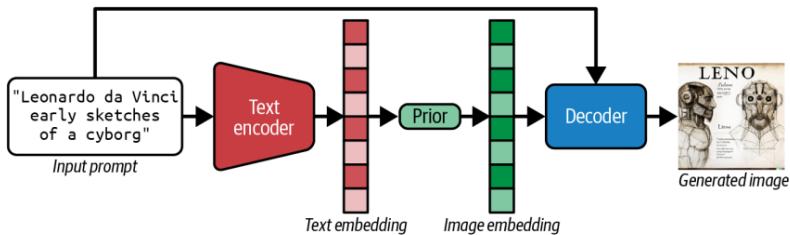
Flamingo



- ▶ A model designed by OpenAI for text-to-image generation.
- ▶ The first version of this model (DALL.E) was released in February 2021¹
- ▶ DALL.E 2 was released in April 2022²
- ▶ DALL.E 3 was released in October 2023

¹<https://arxiv.org/abs/2102.12092>

²<https://arxiv.org/abs/2204.06125>





- ▶ There are three distinct parts: the text encoder, the prior and the decoder.
- ▶ Text is first passed through the text encoder to produce a text embedding vector.
- ▶ This vector is then transformed by the prior to produce an image embedding vector.
- ▶ Finally, this is passed through the decoder, along with the original text, to produce the generated image.



- ▶ The aim of the text encoder is to convert the text prompt into an embedding vector that represents the conceptual meaning of the text prompt within a latent space.
- ▶ In DALL.E 2, the authors do not train the text encoder from scratch, but instead make use of an existing model called Contrastive Language–Image Pre-training (CLIP), also produced by OpenAI.



- ▶ A neural network that learns visual concepts from natural language supervision
- ▶ It uses a technique called contrastive learning
- ▶ It is trained on a dataset of 400 million text-image pairs scraped from the internet - potential example pairs
- ▶ Given an image and list of possible text descriptions, its task is to find the one that actually matches the image.
- ▶ CLIP is closer to a discriminative model, because the final output is a prediction about which text description from a given set matches a given image the closest.

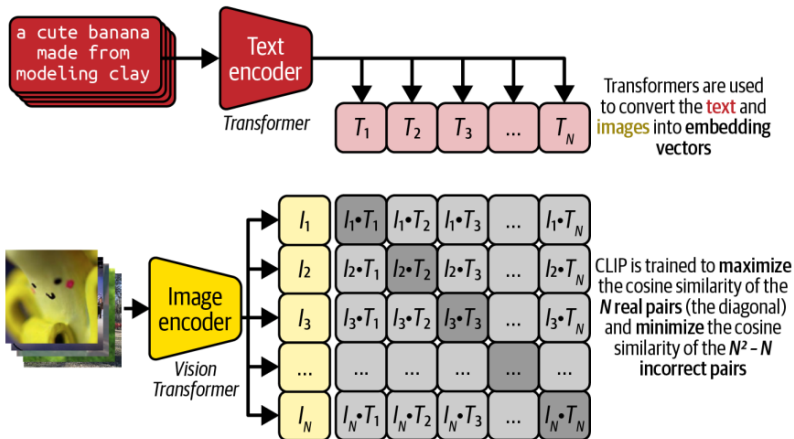
³<https://openai.com/blog/clip/>



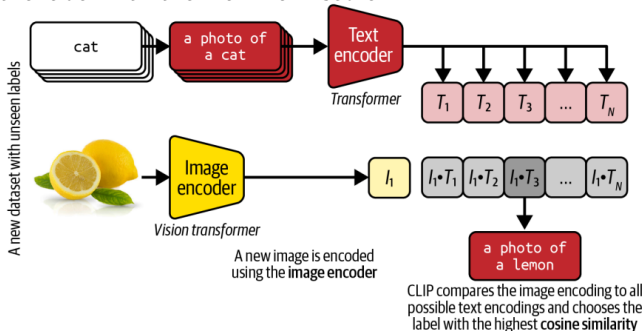
- ▶ We train two neural networks - a **text encoder** that converts text to a text embedding and **an image encoder** that converts an image to an image embedding.
- ▶ Then, given a batch of text-image pairs, we compare all text and image embedding combinations using cosine similarity and train the networks to maximize the score between matching text-image pairs and minimise the score between incorrect text-image pairs.
- ▶ Both the text encoder and image encoder are Transformers - the image encoder is a Vision Transformer (ViT)⁴

⁴<https://arxiv.org/abs/2010.11929>

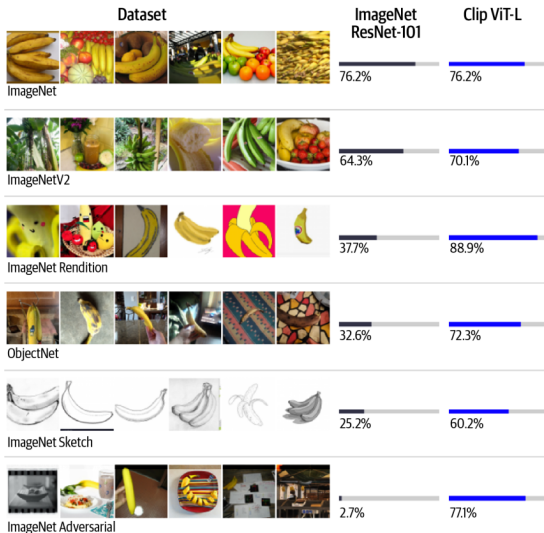
CLIP Training



- ▶ To predict the label of a given image, we can pass it through the CLIP image encoder and calculate the cosine similarity between the image embedding and all possible text embeddings, in order to find the label with the maximum score



CLIP Inference (cont.)





- ▶ Other models that have been trained on a specific dataset to predict a given set of labels, often fail when applied to different datasets with the same labels because the model is highly optimized to the dataset on which it was trained.
- ▶ CLIP is much more robust, as it has learnt a deep conceptual understanding of full text descriptions and images
- ▶ **Question: CLIP is measured on its discriminative ability, so how does it help us to build generative models such as DALL.E?**

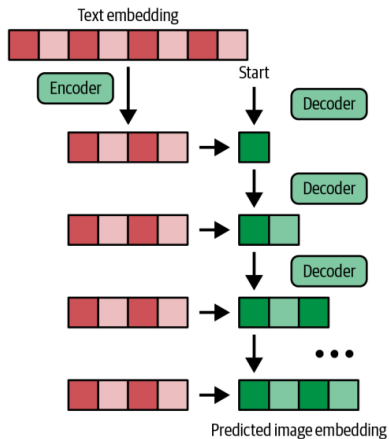


- ▶ The DALL.E authors tried two different methods for training the prior model: an autoregressive model and a diffusion model
- ▶ The the diffusion approach outperforms the autoregressive model and is more computationally efficient.



- ▶ An autoregressive model generates output sequentially, by placing an ordering on the output tokens (e.g., words, pixels) and conditioning the next token on previous tokens. (as in e.g. RNNs or transformers).
- ▶ The autoregressive prior of DALL.E is an encoder-decoder transformer.
- ▶ It is trained to reproduce the CLIP image embedding given a CLIP text embedding.
- ▶ The model is trained on the CLIP text–image pair dataset.
- ▶ The input text embedding is processed by the encoder of the Transformer to produce another representation that is fed to the decoder, alongside the current generated output image embedding.
- ▶ The output is generated one element at a time, using teacher forcing to compare the predicted next element to the actual CLIP image embedding.

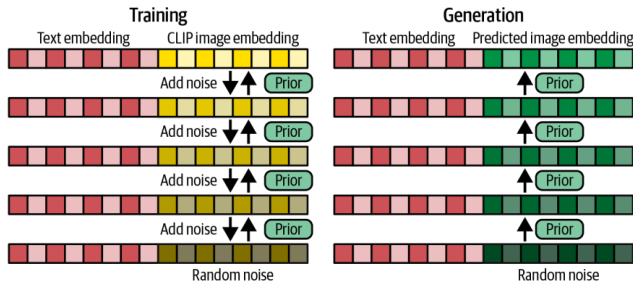
Autoregressive Prior (cont.)





- ▶ The sequential nature of the generation means that the autoregressive model is less computationally efficient than diffusion.
- ▶ In DALL.E 2 a decoder-only Transformer is used as the prior, trained using a diffusion process.
- ▶ During training, each CLIP text and image embedding pair are first concatenated into a single vector.
- ▶ Then, the image embedding is noised over 1,000 timesteps until it is indistinguishable from random noise.
- ▶ The diffusion prior is then trained to predict the denoised image embedding at the previous timestep.
- ▶ The prior has access to the text embedding throughout, so it is able to condition its predictions on this information, gradually transforming the random noise into a predicted CLIP image embedding.

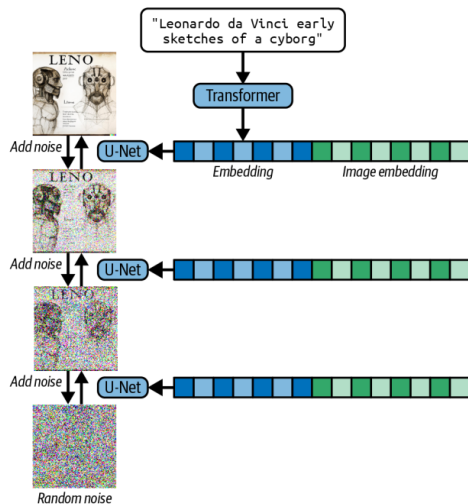
- ▶ The loss function is the average mean-squared error across denoising steps.
- ▶ To generate new image embeddings, we sample a random vector, prepend the relevant text embedding, and pass it through the trained diffusion prior.



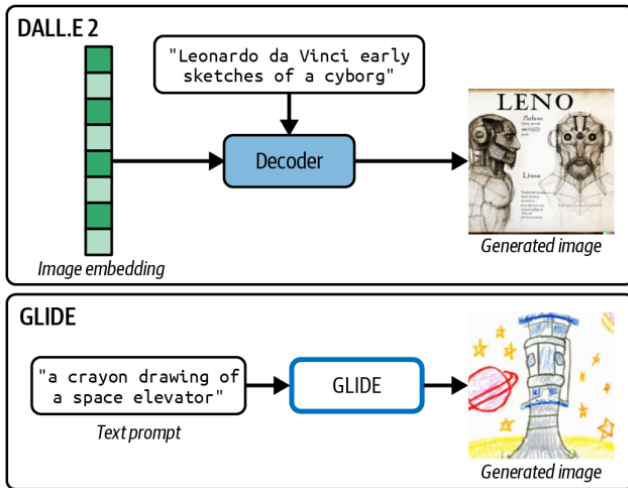


- ▶ Decoder is the part of the model that generates the final image conditioned on the text prompt and the predicted image embedding output by the prior.
- ▶ The architecture and training process of the decoder borrows from an earlier OpenAI paper, published in December 2021, which presented a generative model called Guided Language to Image Diffusion for Generation and Editing (GLIDE).⁵
- ▶ GLIDE is able to generate realistic images from text prompts, in much the same way that DALL.E 2 can.
- ▶ The difference is that GLIDE does not make use of CLIP embeddings, but instead works directly with the raw text prompt

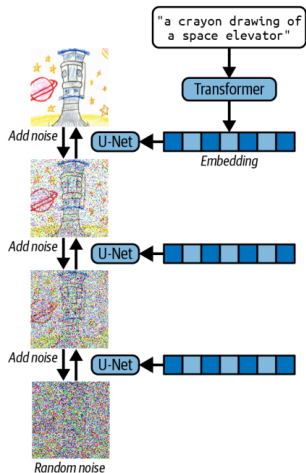
The Decoder (cont.)



⁵<https://arxiv.org/abs/2112.10741>



- ▶ GLIDE is trained as a diffusion model, with U-Net architecture for the denoiser and Transformer architecture for the text encoder.
- ▶ It learns to undo the noise added to an image, guided by the text prompt.
- ▶ A Transformer is used to create an embedding of the input text prompt, which is then used to guide the U-Net throughout the denoising process.
- ▶ Finally, an Upsampler is trained to scale the generated image to $1,024 \times 1,024$ pixels.
- ▶ It is trained on 250 million text–image pairs.





- ▶ The DALL.E 2 decoder still uses the U-Net denoiser and Transformer text encoder architectures, but additionally has the predicted CLIP image embeddings to condition on.
- ▶ As with all diffusion models, to generate a new image, we simply sample some random noise and run this through the U-Net denoiser multiple times, conditioned on the Transformer text encoding and image embedding.
- ▶ The output is a 64×64 -pixel image.

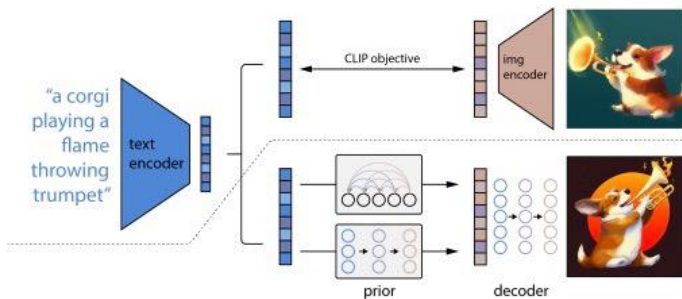
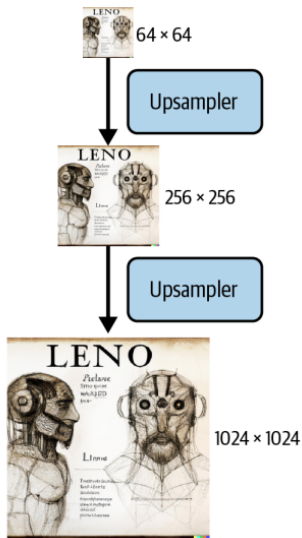


Figure 2: A high-level overview of unCLIP. Above the dotted line, we depict the CLIP training process, through which we learn a joint representation space for text and images. Below the dotted line, we depict our text-to-image generation process: a CLIP text embedding is first fed to an autoregressive or diffusion prior to produce an image embedding, and then this embedding is used to condition a diffusion decoder which produces a final image. Note that the CLIP model is frozen during training of the prior and decoder.



- ▶ The final part of the decoder is the Upsampler (two separate diffusion models).
- ▶ The first diffusion model transforms the image from 64×64 to 256×256 pixels. The second transforms it again, from 256×256 to $1,024 \times 1,024$ pixels.
- ▶ We can work with small images until the final stages of the process, when we apply the Upsamplers.
- ▶ This saves on model parameters and ensures a more efficient upstream training process.

Upsampler (cont.)



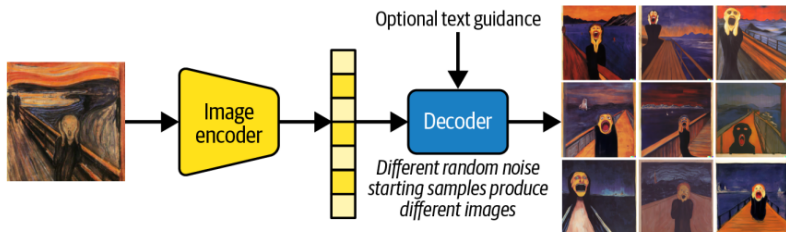


- ▶ Examples: <https://openai.com/dall-e-2>



- ▶ To generate images using the DALL.E 2 decoder we sample an image consisting of pure random noise and then gradually reduce the amount of noise using the denoising diffusion model, conditioned on the provided image embedding.
- ▶ Selecting different initial random noise samples will result in different images.
- ▶ In order to generate variations of a given image, we therefore just need to establish its image embedding to feed to the decoder.
- ▶ We can obtain this using the original CLIP image encoder, which is explicitly designed to convert an image into its CLIP image embedding.

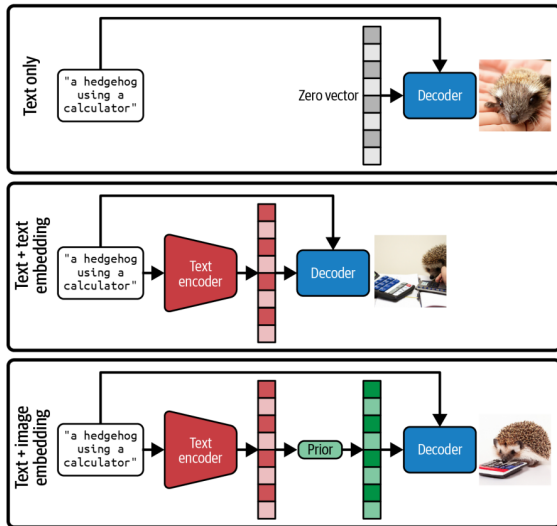
Image Variations (cont.)





- ▶ The purpose of the prior is to provide the decoder with a useful representation of the image to be generated
- ▶ However, this step isn't necessary
- ▶ To test having no prior, the authors tried three different approaches:
 - 1. Feed the decoder only with the text prompt (and a zero vector for the image embedding).
 - 2. Feed the decoder with the text prompt and the text embedding (as if it were an image embedding).
 - 3. Feed the decoder with the text prompt and the image embedding (i.e., the full model).

No prior example



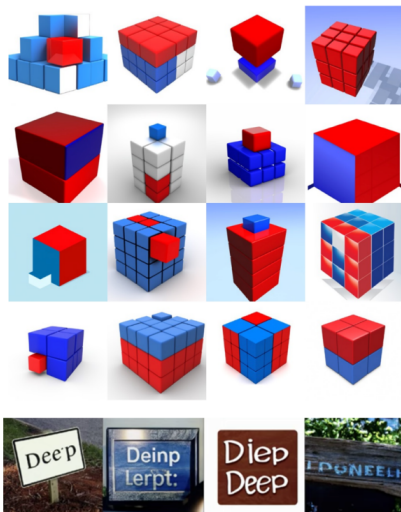


- ▶ When the decoder is starved of image embedding information, it can only produce a rough approximation of the text prompt, missing key information such as the calculator.
- ▶ Using the text embedding as if it were an image embedding performs slightly better, though it is not able to capture the relationship between the hedgehog and the calculator.



- ▶ **Attribute binding** is the ability of a model to understand the relationship between words in a given text prompt, and in particular how attributes relate to objects.
- ▶ For example, the prompt 'A red cube on top of a blue cube' must appear visually distinct from 'A blue cube on top of a red cube.'
- ▶ DALL.E struggles somewhat with this, compared to earlier models such as GLIDE, though the overall quality of generations is better and more diverse.
- ▶ Also, DALL.E 2 is not able to accurately reproduce text—this is probably due to the fact that the CLIP embeddings do not capture spellings, but instead only contain a higher-level representation of the text.

Limitations (cont.)





DALL.E

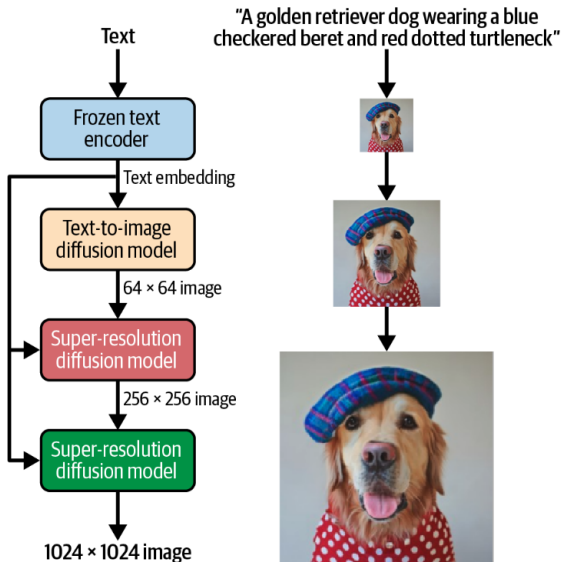
Imagen

Stable Diffusion

Flamingo

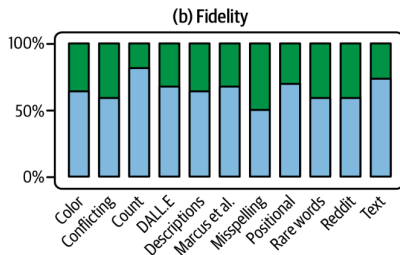
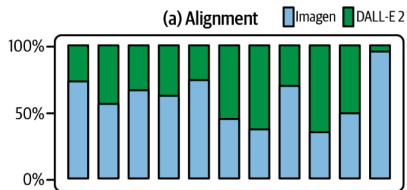


- ▶ The Google Brain team released their own text-to-image model called **Imagen**.





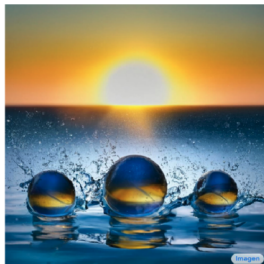
- ▶ The frozen text encoder is the pretrained T5-XXL model, a large encoder-decoder Transformer.
- ▶ Unlike CLIP, this was trained only on text and not images, so it is not a multimodal model.
- ▶ Like DALL.E 2's, Imagen's the decoding diffusion model is based on a U-Net architecture, conditioned on text embeddings.
- ▶ There are several architectural improvements made to the standard U-Net architecture, to produce what the authors call the Efficient U-Net.
- ▶ The Upsampler super-resolution models that take the generated image from 64×64 to $1,024 \times 1,024$ pixels are also diffusion models that continue to use the text embeddings to guide the upsampling process.





- ▶ Whilst Imagen outperforms DALL.E 2 on many of the DrawBench benchmarks, DALL.E 2 provides additional functionalities that are not present in Imagen.
- ▶ For example, because DALL.E 2 utilizes CLIP (a multimodal text–image model), it is able to accept images as input to generate image embeddings.
- ▶ This means DALL.E 2 is able to provide image editing and image variation capabilities.
- ▶ This is not possible with Imagen; the text encoder is a pure text model, so there is no way to input an image.

Examples from Imagen



Three spheres made of glass falling into the ocean. Water is splashing. Sun is setting.



Vines in the shape of text "Imagen" with flowers and butterflies bursting out of an old TV.



A strawberry splashing in the coffee in a mug under the starry sky.



DALL.E

Imagen

Stable Diffusion

Flamingo



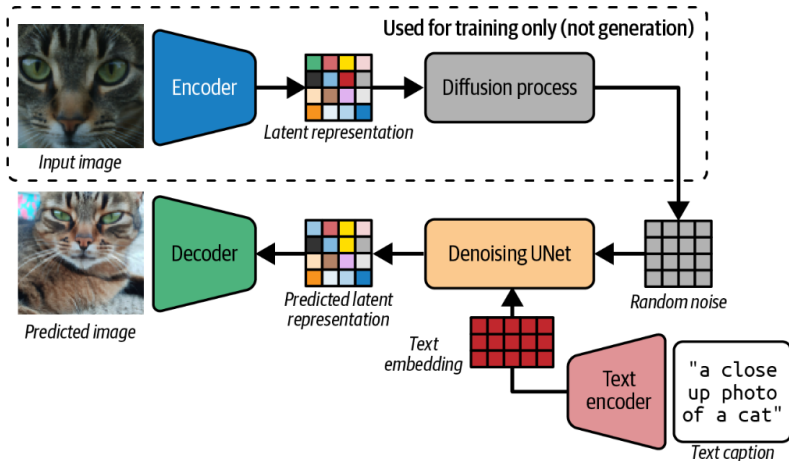
- ▶ Released in August 2022 by Stability AI, in collaboration with the Computer Vision and Learning research group at Ludwig Maximilian University of Munich and Runway.
- ▶ Its code and model weights have been released publicly, through Hugging Face⁶.

⁶<https://huggingface.co/stabilityai/stable-diffusion-xl-base-1.0>



- ▶ The main architectural difference between Stable Diffusion and the text-to-image models discussed previously is that it uses latent diffusion⁷ as its underlying generative model.
- ▶ The key idea is to wrap the diffusion model within an autoencoder, so that the diffusion process operates on a latent space representation of the image rather than the image itself.

Stable Diffusion Architecture (cont.)



⁷<https://arxiv.org/abs/2112.10752>



- ▶ This means that the denoising U-Net model can be kept relatively lightweight, in comparison to U-Net models that operate on full images.
- ▶ The autoencoder handles the heavy lifting of encoding the image detail into latent space and decoding the latent space back to a high-resolution image, leaving the diffusion model to work purely in a latent, conceptual space.
- ▶ This gives a significant speed and performance boost to the training process.
- ▶ The denoising process can also optionally be guided by a text prompt that has been passed through a text encoder.

Examples from Stable Diffusion



"an insect robot preparing a delicious meal"



"a high tech solarpunk utopia in the the Amazon rainforest"



"a small cabin on top of a snowy mountain in the style of Disney, artstation"



https://keras.io/examples/generative/random_walks_with_stable_diffusion/



<https://medium.com/towards-agi/flux-vs-midjourney-vs-dall-e-vs-stable-diffusion-which-ai-image-generator-should-you-choose-30e35c3c680c>



DALL.E

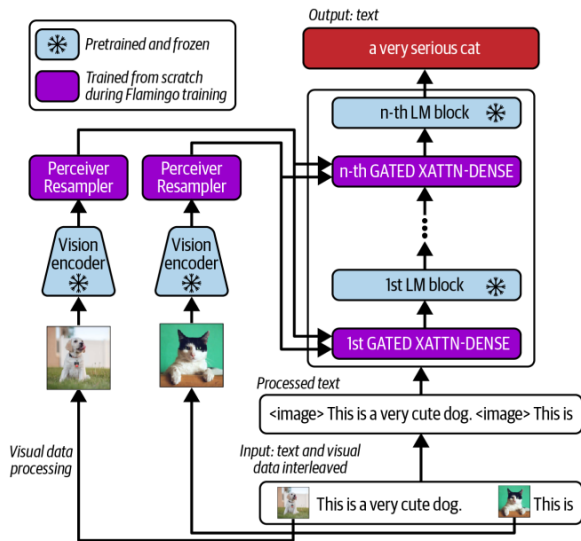
Imagen

Stable Diffusion

Flamingo



- ▶ Introduced by DeepMind in April 2022
- ▶ A family of visual language models (VLMs) that act as a bridge between pre-trained vision-only and language-only models.



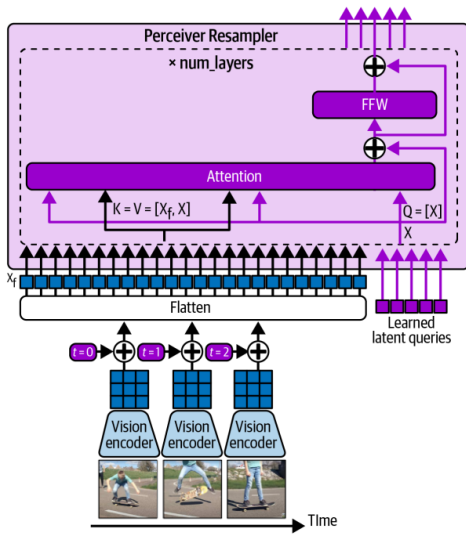


- ▶ Flamingo can accept a combination of text and visual (including videos) data interleaved.
- ▶ The job of the Vision Encoder is to convert the vision data within the input into embedding vectors.
- ▶ The Vision Encoder in Flamingo is a pre-trained Normalizer-Free ResNet (NFNet).
- ▶ The Vision Encoder is trained on image-text pairs using the same contrastive objective as CLIP.
- ▶ The output from the Vision Encoder is a 2D grid of features that then gets flattened to a 1D vector.
- ▶ Video is handled by sampling at 1 frame per second.



- ▶ The output from the Vision Encoder is a vector of variable length (due to the variable input image resolution and the variable number of video frames) and is therefore potentially very long.
- ▶ The Perceiver architecture is specifically designed to efficiently handle long input sequences.
- ▶ Instead of performing self-attention on the full input sequence, it works with a fixed-length latent vector and only uses the input sequence for cross-attention.
- ▶ Specifically, in the Flamingo Perceiver Resampler, the key and value are a concatenation of the input sequence and latent vector and the query is the latent vector alone.
- ▶ The output of the Perceiver Resampler is a fixed-length latent vector that gets passed to the Language Model.

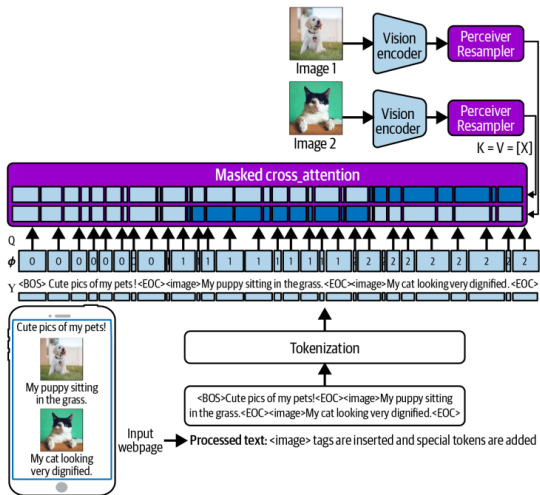
The Perceiver Resampler





- ▶ The Language Model consists of several stacked blocks, in the style of a decoder Transformer
- ▶ Flamingo uses Chinchilla model - a pre-trained relatively small DeepMind model.
- ▶ A key contribution of the Flamingo paper is to show how Chinchilla can be adapted to work with additional vision data (X) that is interspersed with the language data (Y).
- ▶ First the text is processed by replacing vision data (e.g., images) with an `< image >` tag and the text is divided into chunks using the `< EOC >` (end of chunk) tag.
- ▶ Each chunk contains at most one image, which is always at the start of the chunk—i.e., the subsequent text is assumed to relate only to that image.
- ▶ Next, the sequence is tokenized and each token is given an index (ϕ) corresponding to the preceding image index.

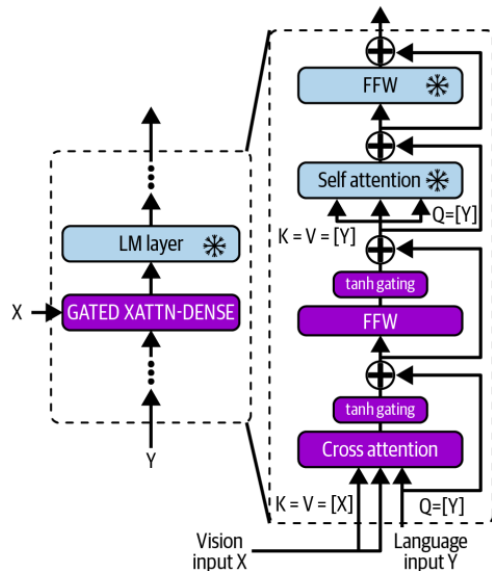
The Language Model cont.



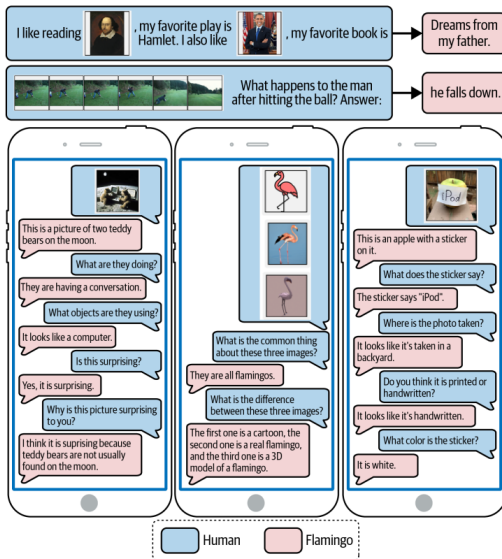


- ▶ The blue LM layer components are frozen layers from Chinchilla—these are not updated during the training process.
- ▶ The purple GATED XATTN-DENSE layers are trained as part of Flamingo and include the masked cross-attention components that blend the language and vision information, as well as subsequent feed-forward (dense) layers.
- ▶ The layer is gated because it passes the output from the cross-attention and feed-forward components through two distinct tanh gates, which are both initialized to zero.
- ▶ Therefore, when the network is initialized, there is no contribution from the GATED XATTN-DENSE layers—the language information is just passed straight through.
- ▶ The alpha gating parameters are learned by the network, to gradually blend in information from the vision data as training progresses.

The Language Model cont.



Flamingo Examples





- ▶ These approaches convert visual features into tokens and prepend them to the language model input.
- ▶ Examples: LLaVA, MiniGPT-4, BLIP-2
- ▶ Mechanism:
 - A vision encoder (often Vision Transformer) extracts image features.
 - A projection layer maps these features into the embedding space of the LLM.
 - The resulting tokens are inserted as prefix tokens before the text.